

A very varm welcome to Varberg!
To our Campus & Tech arena
We hope you have a wonderful time here



Co-funded by
the European Union



Region Halland

Achieving Balance

Bunch o' people. Bunch o' robots. One room.

Goatmire 2026 · James Harton & Gus Workman

Agenda

Hour 1	Intro, assembly & install
Hour 2	How to describe a robot
Hour 3	How to describe your robot
Hour 4	Putting it all together

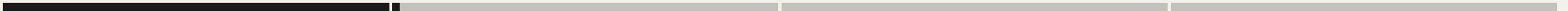
Who we are

JAMES HARTON

- Principal Engineer at Alembic
- Creator of Beam Bots
- Ash core-team member
- More than a decade of Elixir

GUS WORKMAN

- Protolux Electronics
- Nerves core-team member
- Designed and manufactured the Nerves Starter Kit
- ...and the balance bot add-on board



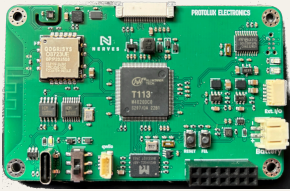
What we're building

- A two-wheeled robot that balances on its own
- A **Nerves Starter Kit** with a **Beam Bots add-on** carrying the motors and the motion sensor
- An e-paper display, four NeoPixels and three buttons
- Its own wifi, so your phone can drive it

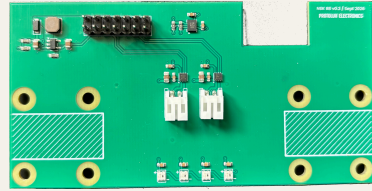
How it came about

- **Goatmire 2025** — I met Gus, who built the conference badges
 - Nerves-powered, and everyone went home with one
 - Same 4" eInk display you've got in front of you
 - Its board grew up into the **Nerves Starter Kit**
 - Lars asked me for a robotics workshop, and I thought of the badge
 - I pitched it to Gus, who was on board before I'd finished asking
-

Make sure you have these parts



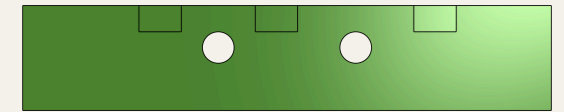
1 × Nerves Starter Kit



1 × Balance bot add-on board



1 × 4" eInk display



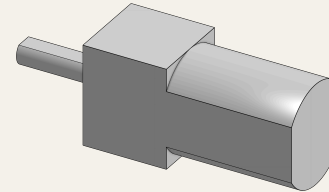
1 × Button board + wire



1 × Chassis half A



1 × Chassis half B



2 × Gearmotors



1 × Battery, tape already on



2 × Wheels

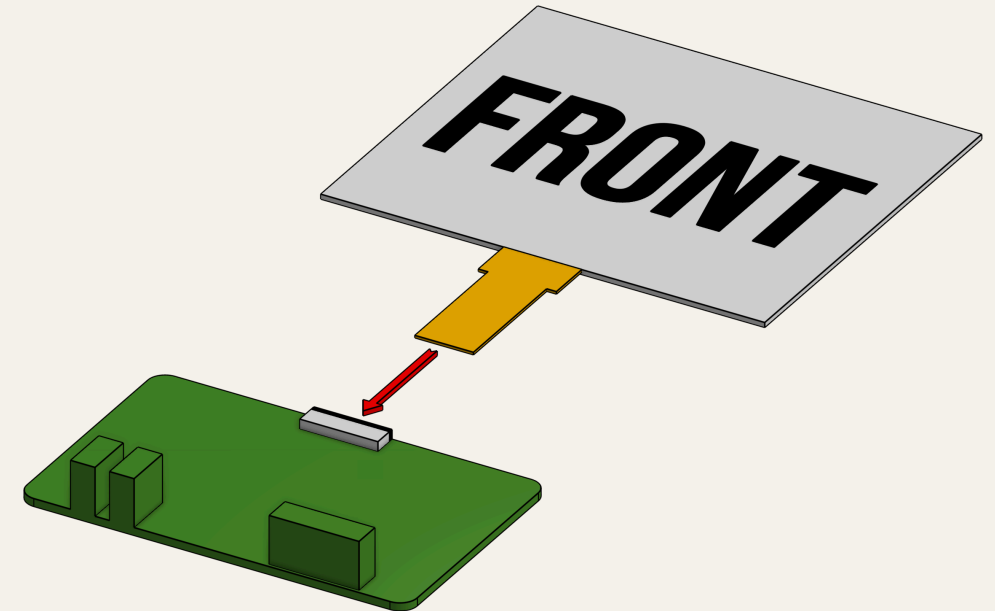


6 × Cable ties

1

Connect the display

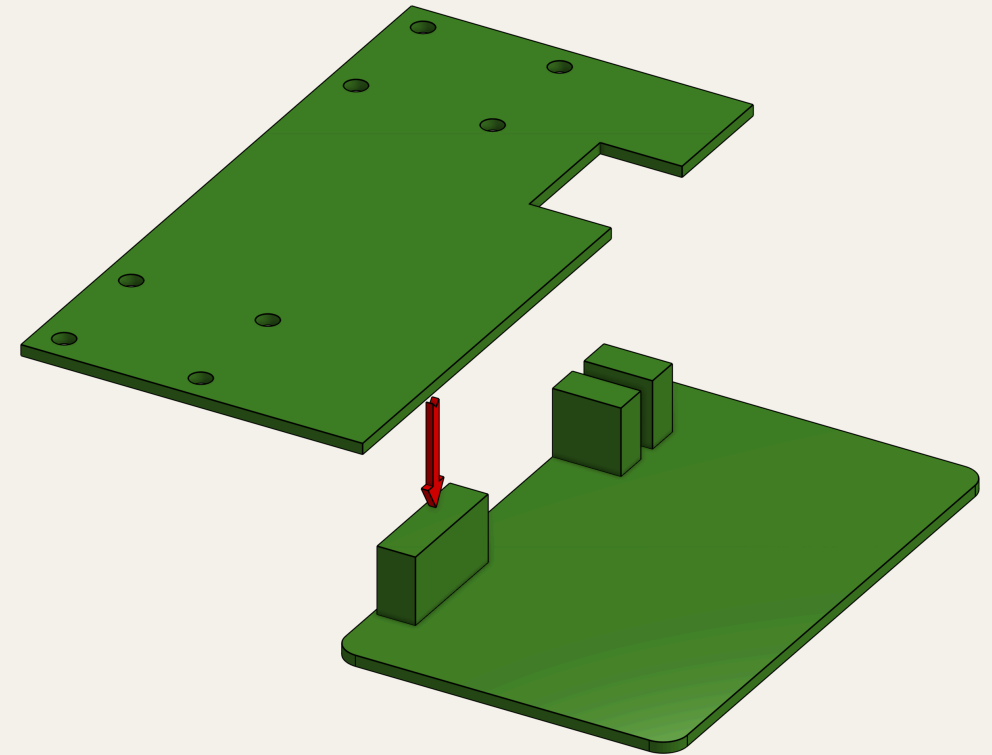
1. Double-check the **Nerves Starter Kit** is switched off — switch position **up**
2. **NSK board face up**, USB connector and pin headers pointing towards you
3. **The 4" eInk display above it**, display side up
4. Carefully open the connector latch, slide the ribbon cable in, close the latch



2

Stack the add-on board

1. Carefully align the **NSK's pin headers** with the socket headers on the balance bot add-on board
2. Gently press the boards together — every pin into its own socket, none bent
3. The **USB-C connector and power switch** stay reachable through the cut-out

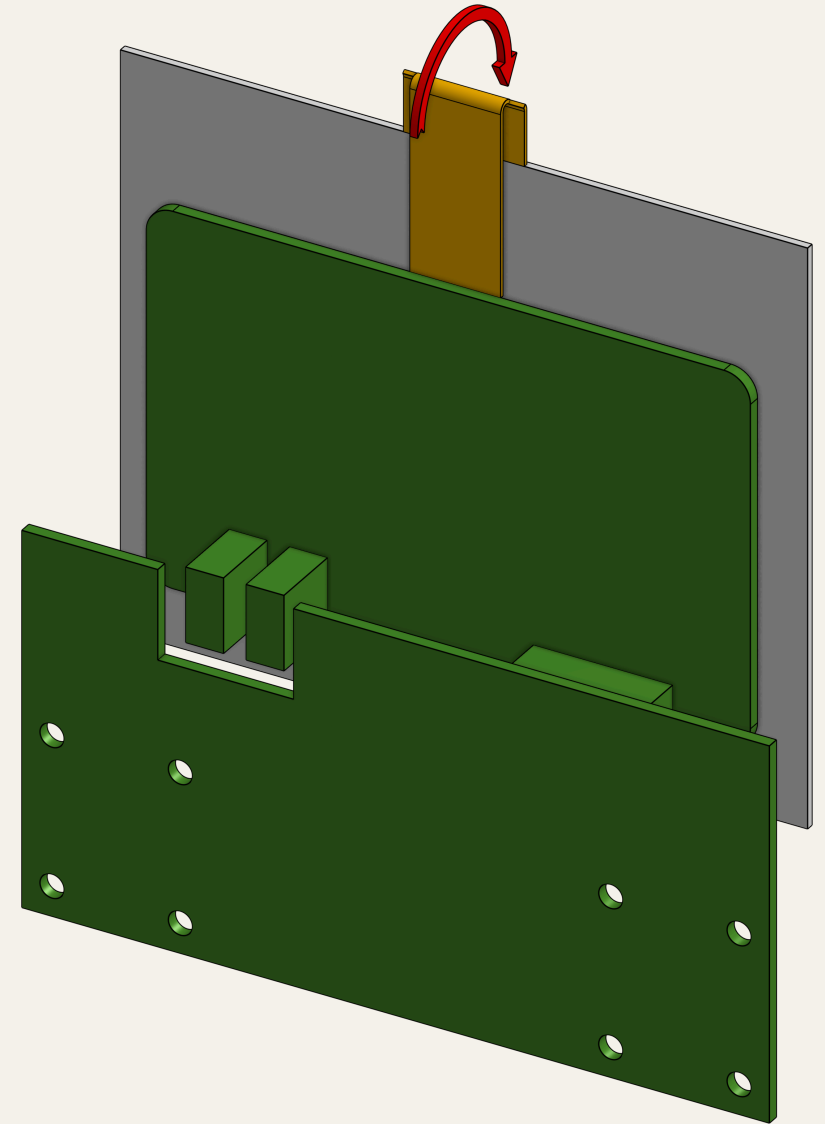


3

Fold the display forward

1. Fold the display over the top edge so it faces **forward**
2. One soft bend over the edge — don't crease it

Yes, the display ends up upside-down. It's meant to be, and the firmware turns the picture the right way up.

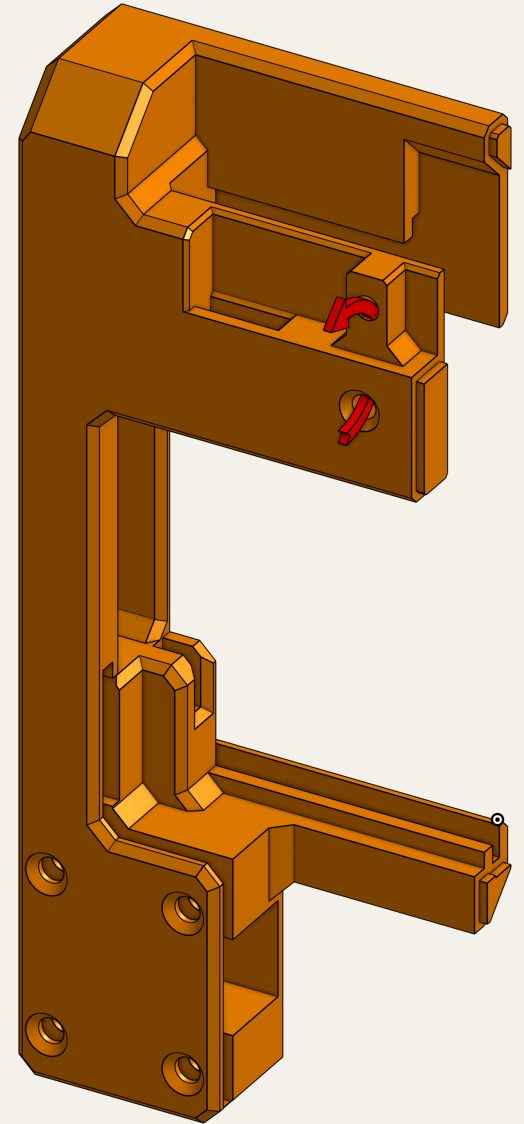


4

Thread the cable ties

1. Find the chassis half marked **A**
2. Feed a cable tie in through the marked hole, bend it around, and poke it back out the top hole
3. **Do not close it**
4. Repeat on the half marked **B**

You'll pull these tight much later. Close one now and you'll have to cut it off and start again.

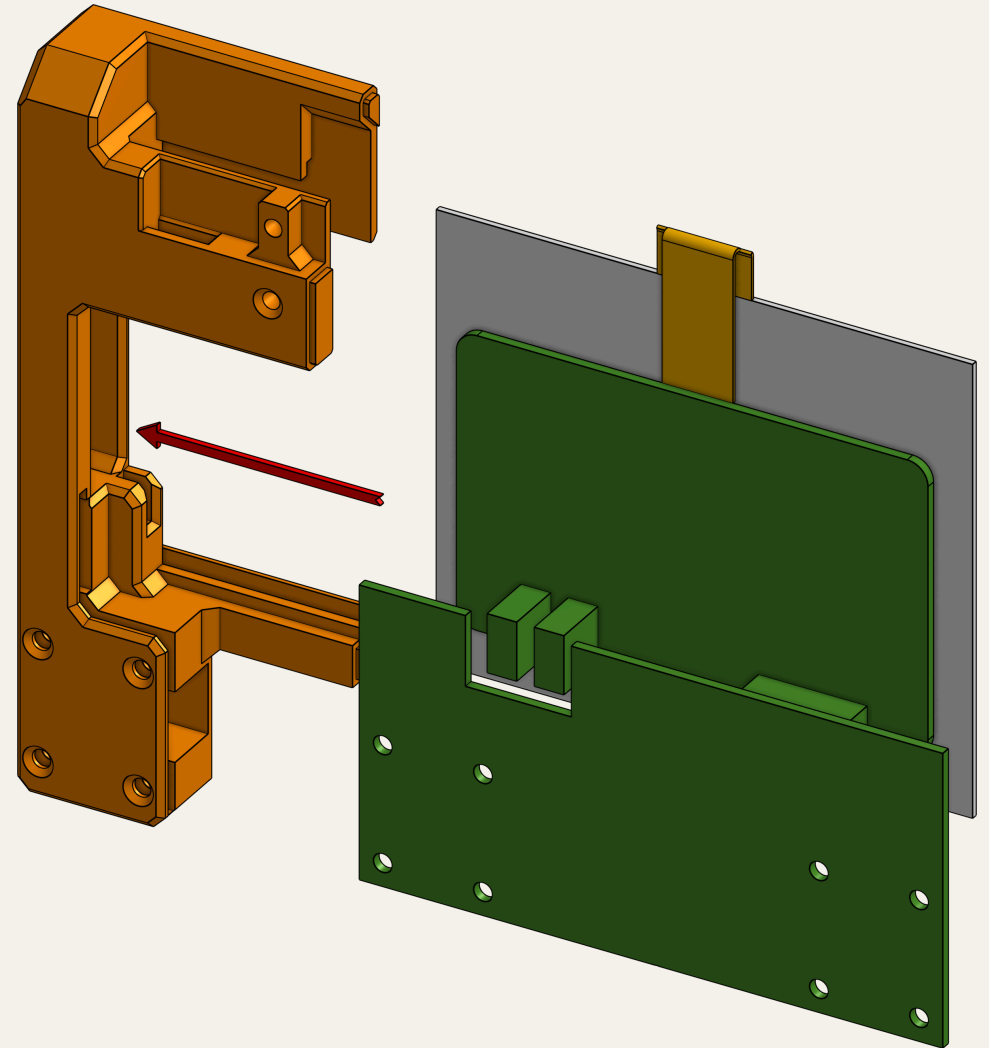


5

Slide the stack into A

1. Slide the display and boards into their slots on part **A**
2. It's fiddly, so take your time. It'll go together — don't force it

Don't put any stress on the display's ribbon cable.

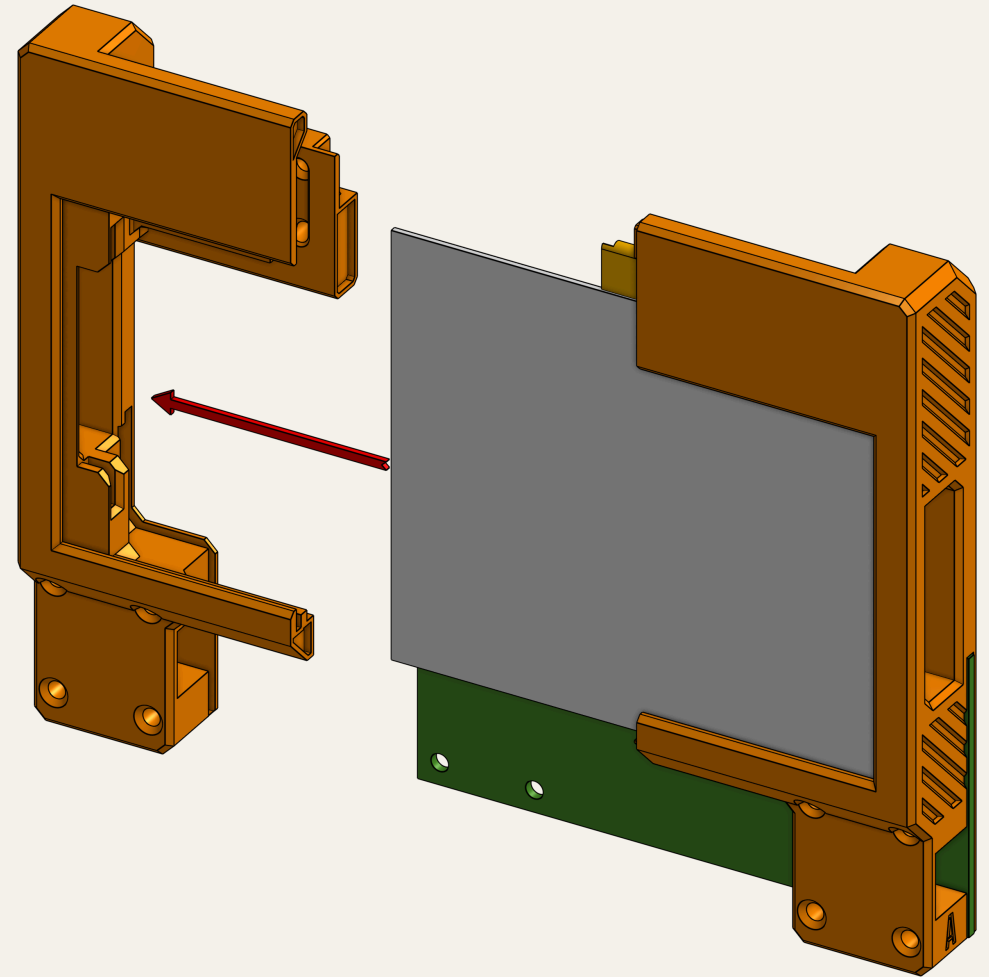


6

Close it up with B

1. Repeat step 5 for side **B**, bringing it onto the other side of the stack
2. Same again — watch the alignment and take your time

Don't pinch the display's ribbon cable between the two halves as they come together.

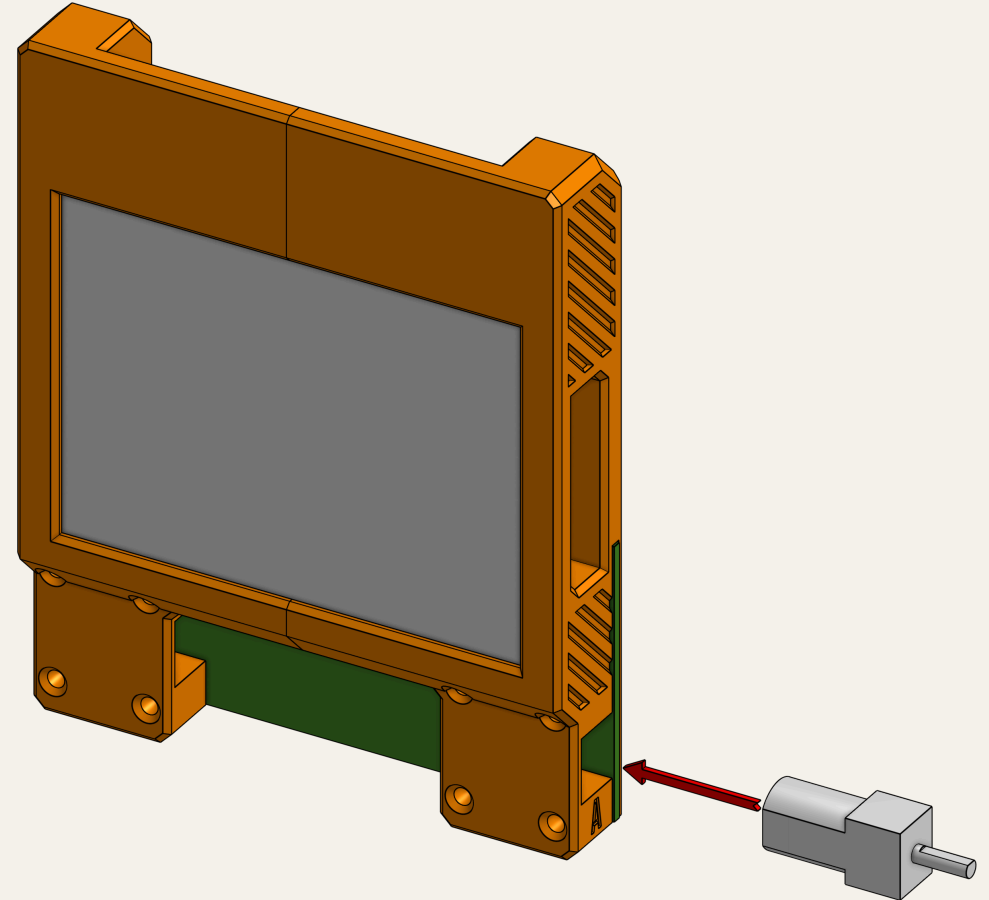


7

Fit the first motor

1. Feed the **motor wires** through the mounting holes first
2. Feed the motor through until it sits **flush with the edge of the chassis**
3. Plug it into the add-on board, using the connector that **faces towards that motor**

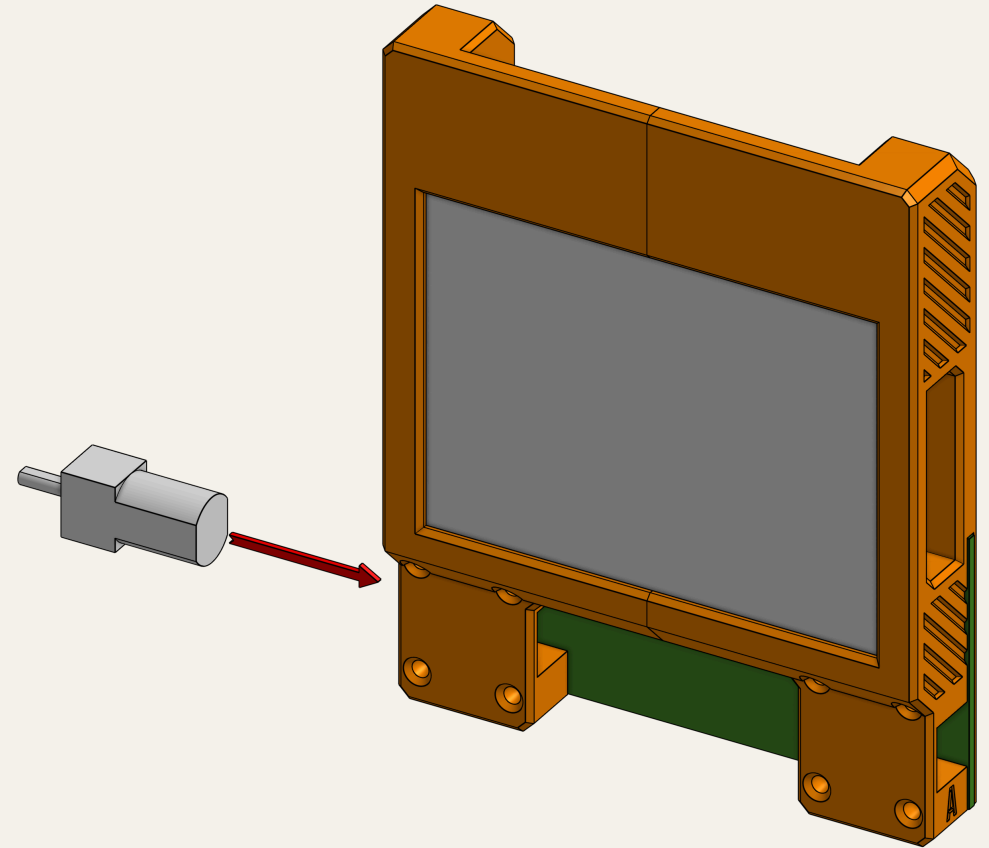
Each motor goes to the connector on its own side.
Cross them over and you'll swap the robot's left and right.



8

Fit the second motor

1. Repeat step 7 for side **B**
2. Both motors flush with the chassis, and each plugged into the connector on its own side

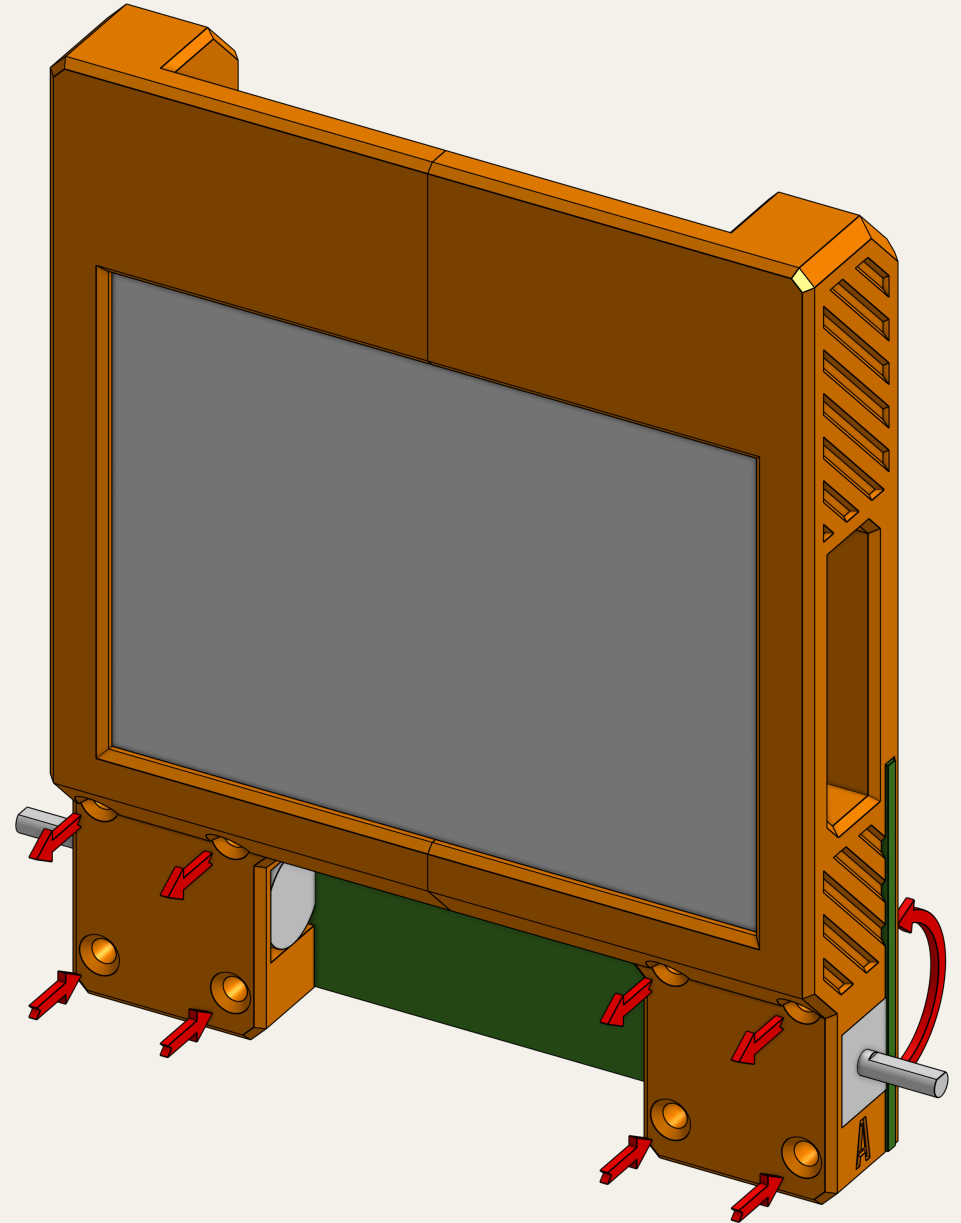


9

Zip-tie the motors

1. **Two cable ties on each side**, four in total
2. In from the **front** through the lower hole, then back out through the **upper hole**
3. Repeat four times
4. The motors are a snug fit already, but pull each tie quite tight — you want **no wobble or play** at all
5. Trim the ends off the ties

Not the ties from step 4 — those stay threaded and open.

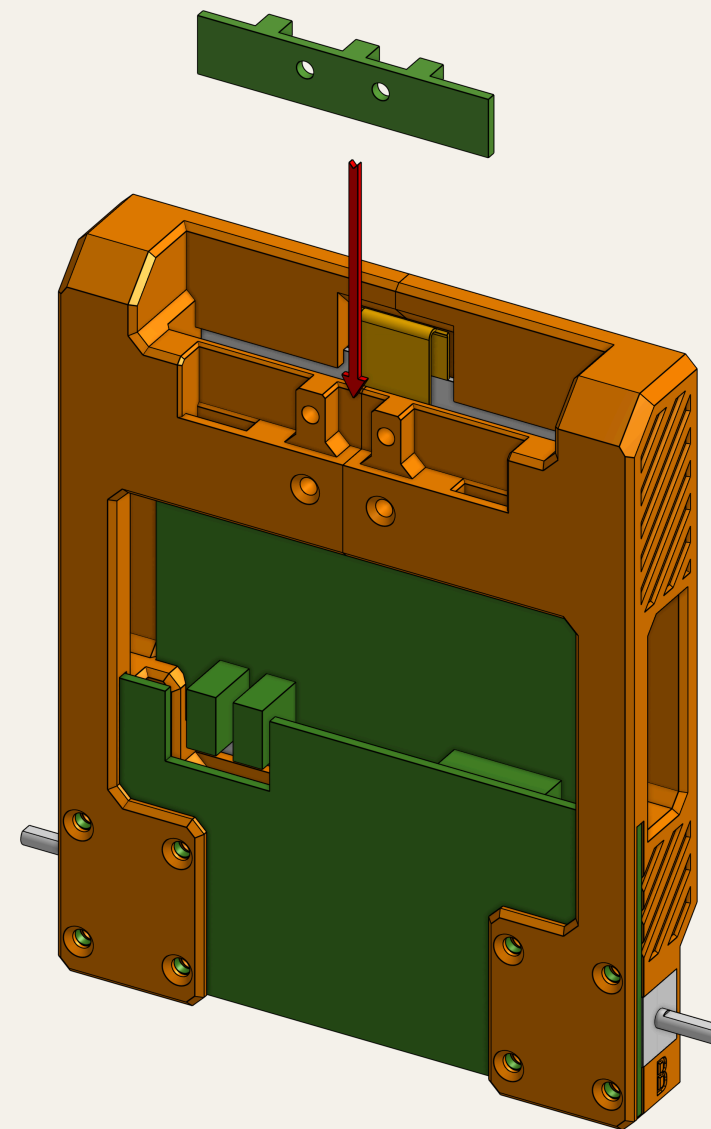


10

Fit the button board

1. Take the **button board** and thread its cable down through the slot in side **B**
2. Plug the cable into the connector on the **edge of the NSK board**

Tweezers or pliers help here — you reach the connector through the cut-outs in the side of the chassis.

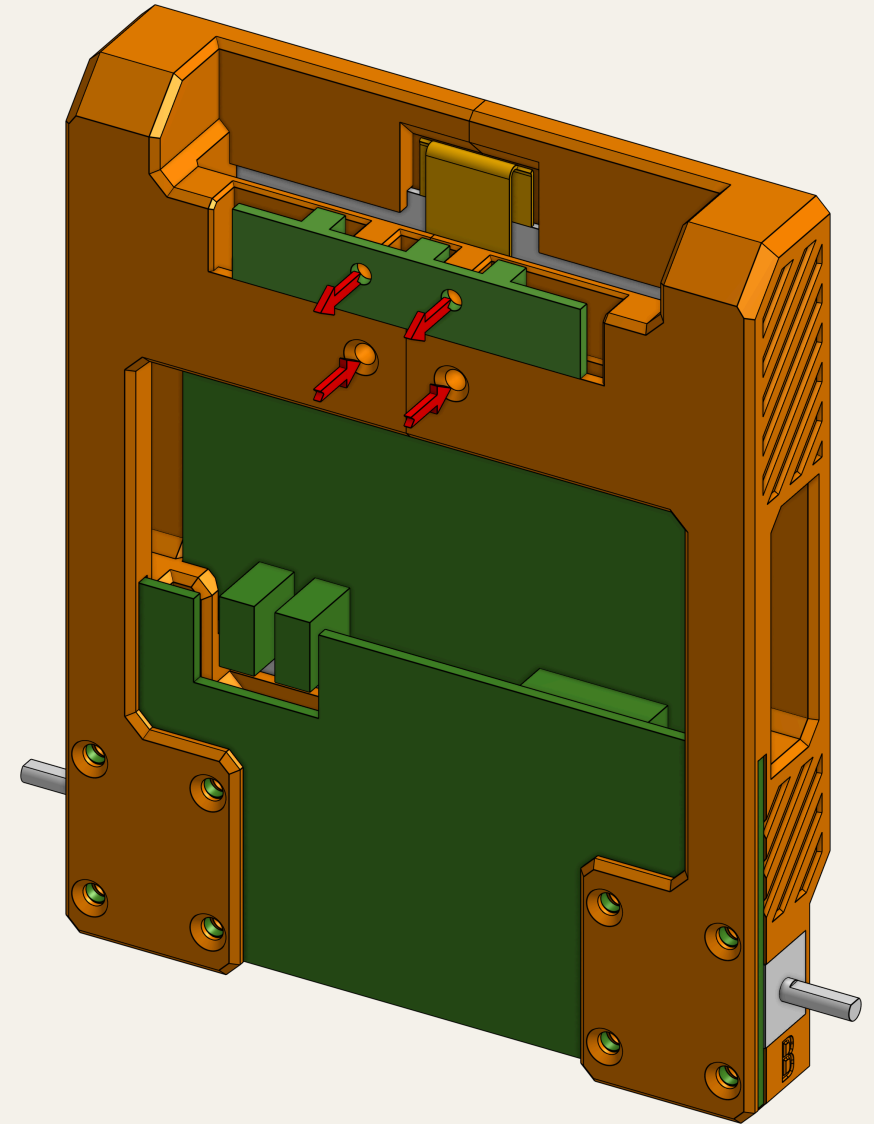


11

Secure the button board

1. Thread the cable ties from **step 4** through the holes on the button board
2. Seat the board on the chassis **shelf**, buttons sitting in the recesses
3. Close and tighten the ties — just enough to hold it without wobble or vibration
4. Trim the ends off the ties

You should be able to reach and press the buttons from the top edge of the board.

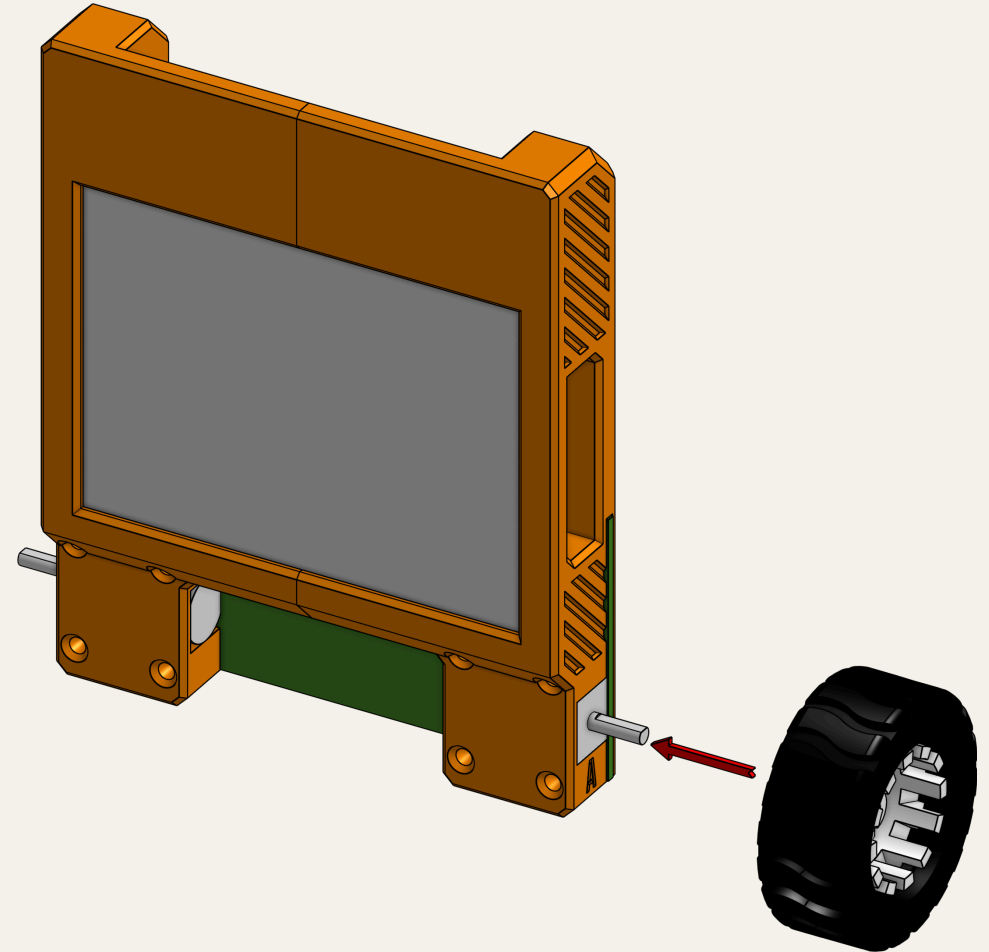


12

Wheels on

1. Press the wheel all the way onto the motor shaft — a flat on the shaft lines up with a flat in the wheel's axle hole
2. Carefully pull it back off a fraction, just until the wheel **spins free** without rubbing on the chassis
3. Repeat on the other side

The wheels go on "**backwards**" — dished side facing out, because the motor shafts are short. It looks wrong. It is right. Leave it that way.

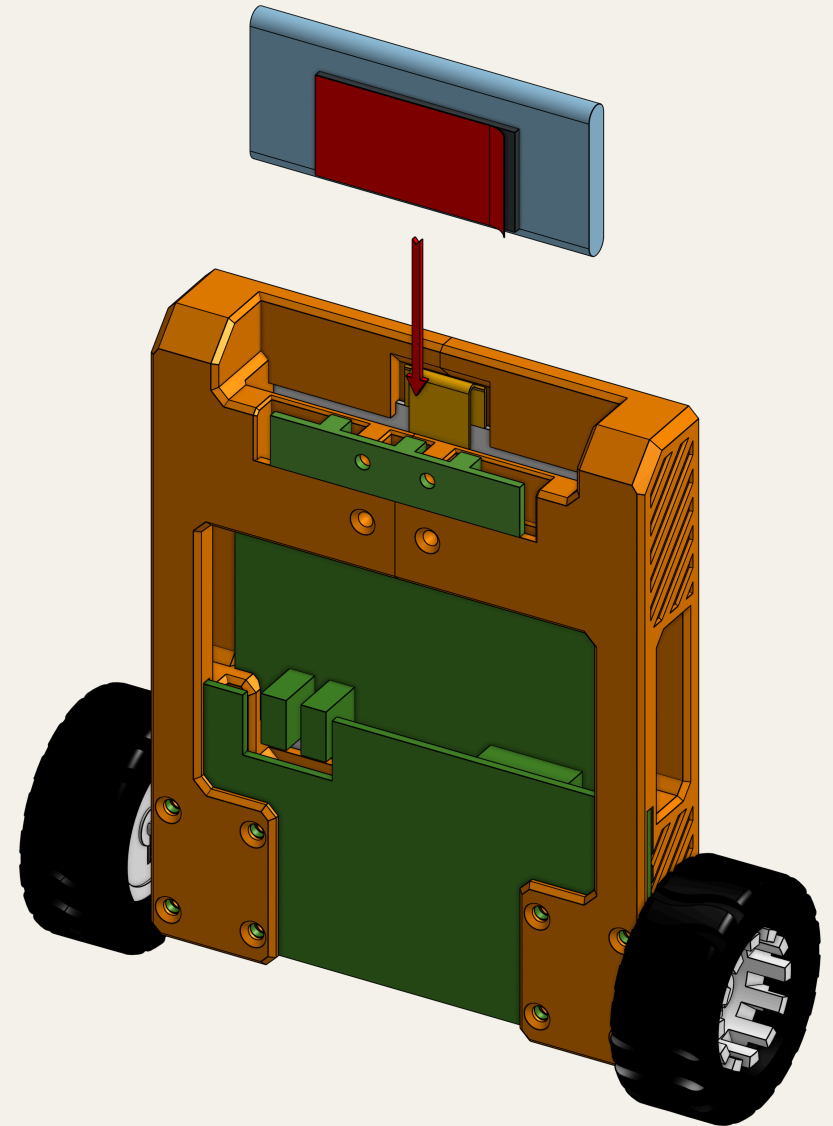


13

Fit the battery

1. Thread the battery cable down the thin gap in the chassis behind the screen. The connector is chunky, but there is a gap at the centre to thread it through
2. Slide the battery into the same slot, **tape facing towards the back**, and feel how it sits with about **10 mm** protruding

Be very careful not to crush or crease the display's ribbon cable as the battery slides in. That's why the tape is on the back, not the front.

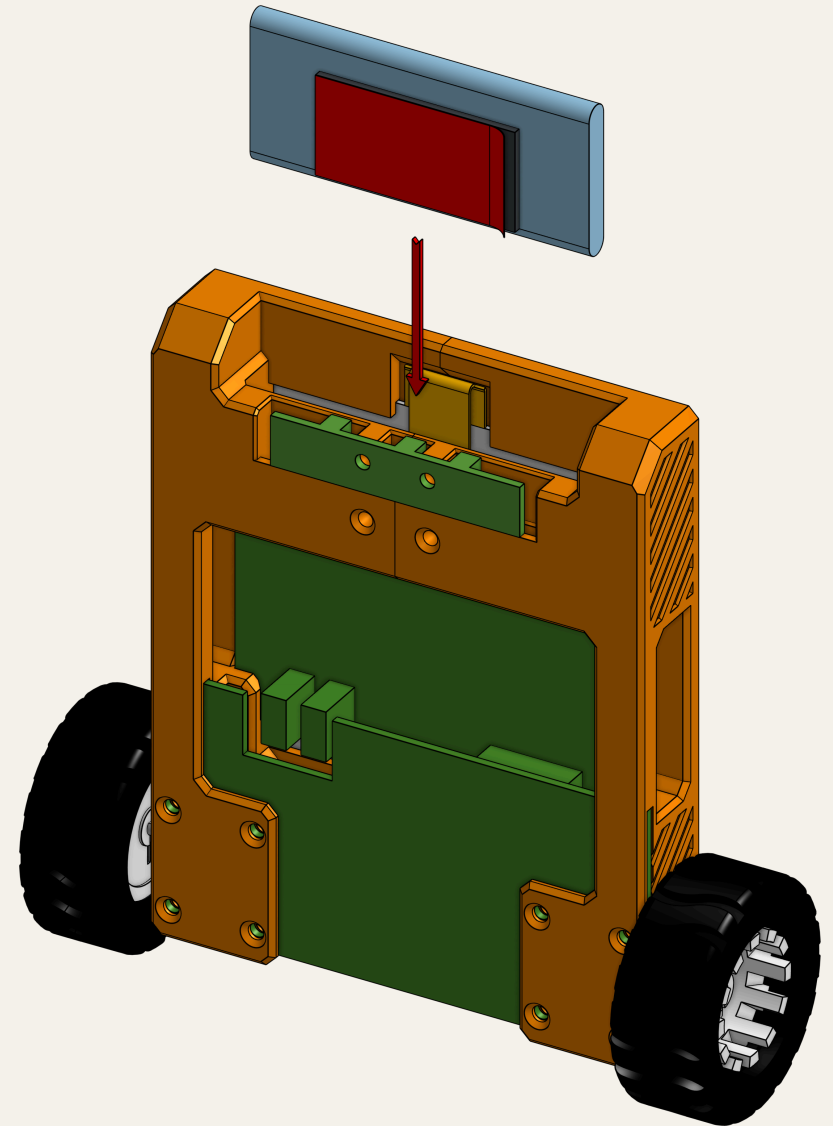


13 cont.

Stick it down and connect it

1. Happy with the position? Slide it back out far enough to peel the covering off the tape
2. Slide it back in and press gently towards the rear of the bot
3. Connect it to the connector on the side of the NSK board, **next to the button board's** — tweezers through the side cut-out again if you need them

That's a lithium polymer cell — dangerous if you crush or puncture it. Tweezers on the connector, fingers on the battery.



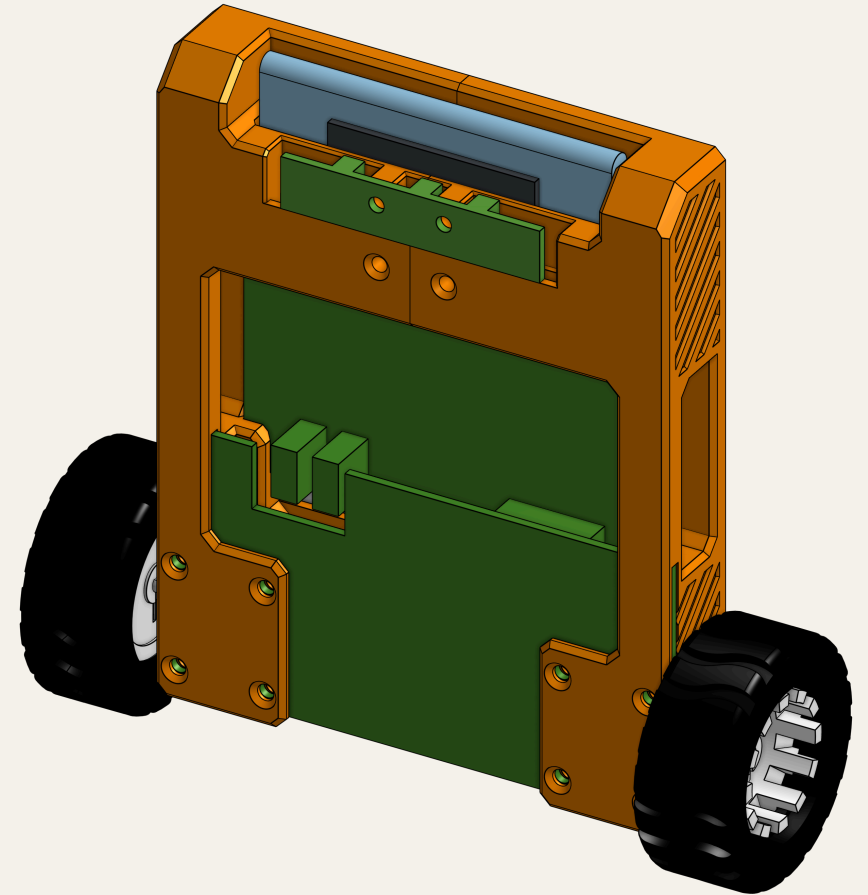
14

That's a robot

Before you put it down:

- Is the battery seated, and its cable connected?
- Is the button board firm, with the tie ends trimmed?
- Do both wheels spin free without rubbing?
- Can you press the buttons from the top edge?

| Give it a gentle shake. Nothing should rattle or shift.



HOUR 1 • BEFORE THE PROJECT

You'll need an SSH key

```
$ ls ~/.ssh/id_*.pub || ssh-keygen -t ed25519
```

Nerves bakes your public key into the firmware so you can log into the robot. No key, no build.

Host tools

```
# macOS  
$ brew install fwup libusb dtc zlib pkg-config  
# ...and on Apple Silicon, if a build says libfdt.h is missing  
$ export DTC_PREFIX=$(brew --prefix dtc)
```

```
# Debian / Ubuntu  
$ sudo apt install fwup libusb-1.0-0-dev libfdt-dev zlib1g-dev pkg-config
```

fwup writes firmware. The rest build the USB tools.

HOURL 1 • INSTALL

Set up your project

```
$ mix archive.install hex nerves_bootstrap  
$ mix archive.install hex igniter_new  
$ mix igniter.new eunice --with nerves.new \  
    --with-args="--target trellis" --install bb_nsk  
$ cd eunice
```

Every robot needs a great name. Swap eunice for yours.

HOURL 2

How to describe a robot

Before we describe yours, let's look at what Beam Bots gives you to describe a robot with.

Open `lib/eunice/robot.ex`

- Around a hundred lines, and the comments say where the numbers came from
- Three commands, and a body with real dimensions and a real mass
- Nothing that senses. Nothing that moves. Not yet

There's a lot in there. Don't worry — we're about to go through all of it.



What a robot is made of

THE SHAPE

- **links** — the rigid bits
- **joints** — how they move

THE SENSES

- **sensors** — what they measure
- **estimators** — working it out

THE DOING

- **actuators** — move a joint
- **controllers** — decide what to do

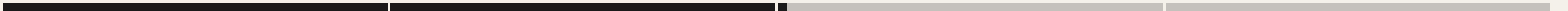
THE RULES

- **commands** — what you can ask
- **states** — when you can ask

parameters — configuration values that you can change at runtime

Kinematics

- Where every part of the robot is, relative to every other part
- A chain of **links**, connected by **joints**
- Every link carries a frame; every joint says how two frames may move
- Tell it where the joints are, and it can tell you where anything is



It starts at the world

```
link :world do
  joint :ground do
    type :planar          # where it is on the floor

    link :ground_contact do
      joint :lean do
        type :revolute    # how far it's tipped over
```

The topology describes a whole kinematic system, not just the robot.

Links

```
link :base_link do
  visual do
    box x: ~u(20.7 millimeter), y: ~u(99 millimeter), z: ~u(124 millimeter)
  end

  inertial do
    mass ~u(138 gram)
    origin x: ~u(2.4 millimeter), z: ~u(45.5 millimeter)
  end
end
```

A rigid chunk of robot. It doesn't bend, and everything else hangs off one.

What goes in a link

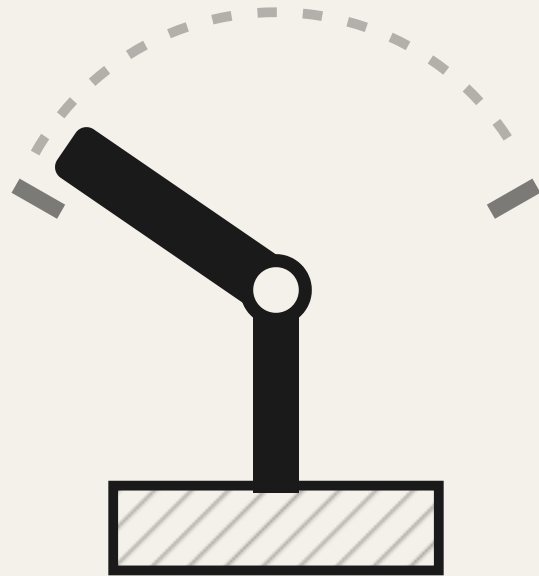
- **inertial** — its mass, and how that mass is spread about
 - **visual** — what it looks like
 - **collision** — the shape used for working out what it bumps into
 - **sensor** — anything measuring from this spot
 - **estimator** — anything working things out from what's measured
 - **joint** — and how the next link hangs off this one
-

Joints

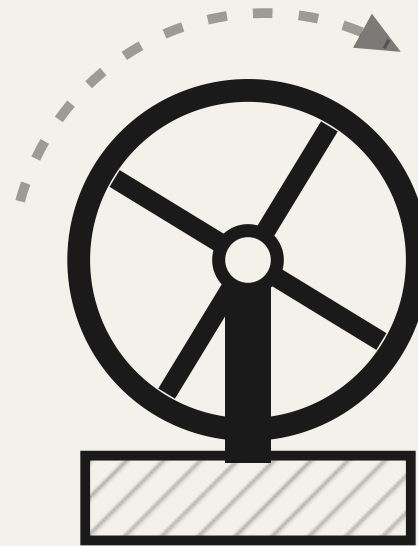
```
joint :left_wheel_joint do  
  type :continuous  
  origin y: ~u(55 millimeter)  
end
```

How one link moves relative to another. Six types to choose from.

Rotation, with and without stops

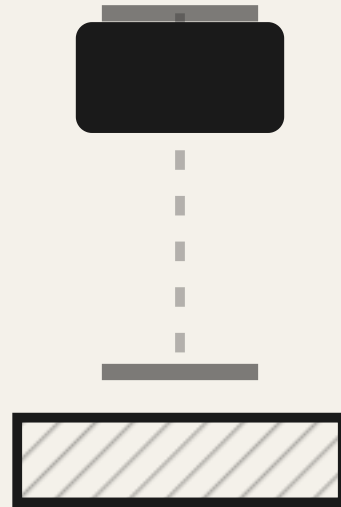


:revolute
stops at its limits

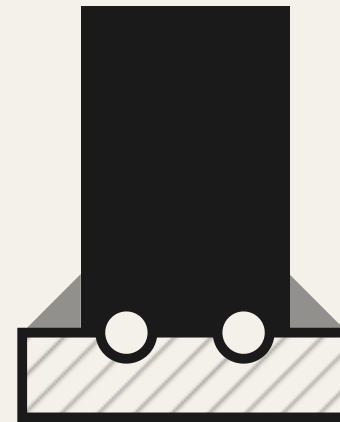


:continuous
keeps going round

Sliding, and not moving at all

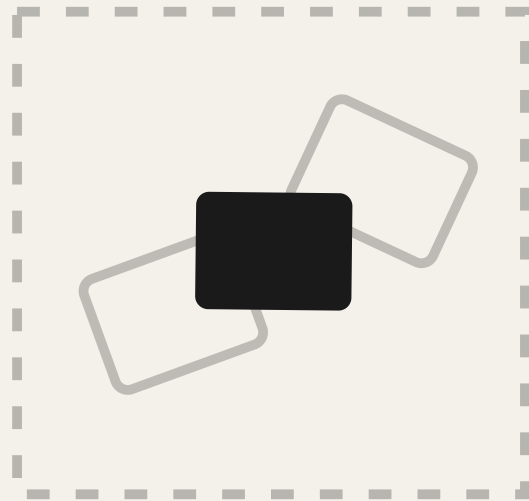


:prismatic
slides along an axis



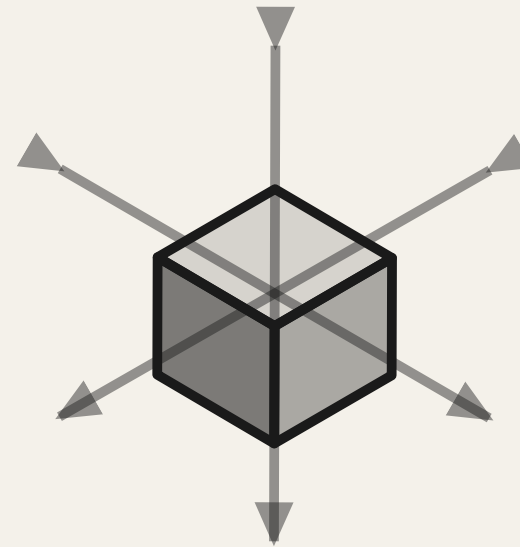
:fixed
doesn't move, on purpose

More than one freedom at a time



:planar

slide, slide and turn



:floating

nothing held at all

Sensors

```
sensor :imu, {BB.Sensor.BMI323,  
  bus: "i2c-0", address: 0x68}
```

A process that reads some hardware and publishes what it measured.

Estimators

```
estimator :orientation,  
  {BB.Estimator.Ahrs.Mahony, kp: 0.4}
```

Turns measurements into the thing you actually wanted.

Actuators

```
actuator :left_wheel, {Eunice.Wheel,  
  forward_pwm: 3, enable_pin: "PE6"}
```

Hangs off a joint, takes a command, and makes the hardware move.

Controllers

```
controller :balancer,  
  {Eunice.Balance.Controller, gain: 180.0}
```

Listens, decides, commands. This is where your loop lives.

Commands

```
command :stand do  
  handler Eunice.Command.Stand  
  allowed_states [:idle, :fallen]  
end
```

The things you're allowed to ask the robot to do.

States

```
state :balancing,  
      doc: "Actively holding itself upright"
```

Where the robot is in its own life, and which commands are legal there.

Parameters

```
parameters do
  group :balance do
    param :proportional_gain,
      type: :float, default: 180.0,
      min: 0.0, max: 250.0
  end
end
```

Numbers with a type, a default and bounds. Groups give them a path.

Change them while it's running

```
iex> BB.Parameter.get!(Eunice.Robot, [:balance, :proportional_gain])  
180.0
```

```
iex> BB.Parameter.set(Eunice.Robot, [:balance, :proportional_gain], 200.0)  
:ok
```

No rebuild. No reboot. Not even a pause.

A robot is a supervision tree

- Each **sensor**, **actuator**, **estimator** and **controller** is a process
- The DSL describes the machine; BB starts and supervises it
- A sensor that falls over gets restarted, and the rest keeps running
- Which means you can run a robot with **no robot attached**

Start your robot

```
$ iex -S mix  
iex> Eunice.Robot.robot()  
iex> Eunice.Robot.arm()  
iex> Eunice.Robot.disarm()
```

No hardware. No firmware. It's still a robot.

Stretch goal — Mauricio's dashboard, for a robot that can't do anything yet:

```
$ mix igniter.install bb_tui --nerves  
$ mix bb.tui --robot Eunice.Robot
```

Where you should be

- ❑ Does `iex -S mix` start without complaining?
- ❑ Does `Eunice.Robot.robot()` hand you back a struct?
- ❑ Does `arm()` hand you back `{:ok, pid}`, and `disarm()` after it?
- ❑ What happens if you `arm()` twice? Why?

HOUR 3

How to describe your robot

Two commands, and it gains wheels and a sense of which way is up.

HOUR 3 • RUN THIS

Give it wheels

```
$ mix bb_nsk.add_wheels
```

Two joints, two actuators, and a parameter for the motor deadband.

A joint that can be driven

```
joint :left_wheel_joint do
  type :continuous
  axis roll: ~u(-90 degree)
  origin y: ~u(55 millimeter)

  actuator :left_wheel,
    {BB.NSK.Wheel,
     forward_pwm: 3, reverse_pwm: 2, enable_pin: "PE6",
     deadband: param([:motor, :deadband])}
end
```

Everything from hour two, in one block.

The motor ignores small commands

```
# on the actuator, from the last slide
deadband: param([:motor, :deadband])

# and the parameter it points at
group :motor do
  param :deadband, type: :float, default: 0.01, min: 0.0, max: 0.9
end
```

Below about 1% duty the motor won't turn at all — and a balance loop lives right there, near zero.

Left and right belong to the robot

- Stand in front of it and its left is on **your right**
- The PWM channels were confirmed by driving each wheel and watching
- We have got this wrong **three times**

If a wheel turns the wrong way, drive it and watch. Don't reason it out from the schematic.



HOUR 3 • RUN THIS

Give it a sense of balance

```
$ mix bb_nsk.add_imu
```

The motion sensor, a filter to make sense of it, and two sensors that turn the answer into joint positions.

Telling it where the sensor is

- The chip measures everything relative to **itself** — its own up and forward
- It's soldered on sideways, so its idea of up isn't the robot's
- A sensor can't say where it is — it borrows that from its link
- So the mounting is its own little chain: **move** it, then **turn** it

One joint doing both would turn first and move second, and put the chip 29.5mm in front of the wheels instead of 29.5mm above them.

Neither sensor is good enough

- The **accelerometer** feels gravity, so it knows which way down is
- ...but it also feels the robot accelerating, and can't tell the two apart
- The **gyroscope** measures turning — quick and smooth, but its errors add up

Its zero-rate offset is spec'd at up to **1°/s**. Integrate that and you're 60 degrees adrift in a minute.



So use both

- Follow the **gyroscope** for what's happening right now
- Lean on the **accelerometer** slowly, to pull the drift back
- Each one covers the other's weakness

That's all an **AHRS filter** is — a recipe for blending the two into one answer.



Mahony, not Madgwick

- Both have a dial for **how hard the accelerometer may pull**
- Here it has to be **small** — this robot accelerates most when it can least afford a wrong answer
- But turn that dial down and gyroscope drift clears slowly too

Madgwick has only that one dial. Mahony has a second one, just for the drift.



The chip, and the filter inside it

```
sensor :imu,  
  {BB.Sensor.BMI323,  
    bus: "i2c-0",  
    address: 0x68,  
    gyroscope_range: 500,  
    publish_rate: param([:sampling, :publish_rate])} do  
  estimator :orientation,  
    {BB.Estimator.Ahrs.Mahony,  
      kp: param([:ahrs, :kp]), ki: param([:ahrs, :ki])}  
end
```

The estimator sits **inside** the sensor, because it's part of that sensor's answer.

HOUR 3

Now let's put some software on this little guy

Everything so far has been on your laptop. Time to make it real.

HOUR 3 • START THIS NOW

Build the firmware

```
$ MIX_TARGET=trellis mix deps.get  
$ MIX_TARGET=trellis mix firmware
```

About five minutes from cold. Start it now — there's nothing else to install first.

Getting it onto a blank board

- Nothing on the eMMC yet, so the first flash goes over **USB**
- Hold the tiny **FEL** button while you switch the power on
- On Linux, `mix nsk.fel` does that for you. On a Mac, use the button
- `mix nsk.ums` offers the eMMC up as a **USB disk** for `mix burn`

No hubs or docking stations — a plain USB cable, straight into your machine. On a Mac, answer **Ignore** to the unreadable-disk prompt.

HOUR 3 • ONCE THE BUILD FINISHES

Flash it

```
$ mix nsk.fel    # Linux – on a Mac, hold FEL and switch on instead  
$ mix nsk.ums  
$ MIX_TARGET=trellis mix burn
```

Plain USB cable, straight into your laptop. On a Mac, answer **Ignore** to the unreadable-disk prompt.

HOUR 3 • ONCE IT'S BOOTED

Make a wheel turn

```
$ ssh nerves.local  
iex> Eunice.Robot.arm()  
iex> BB.Actuator.velocity(Eunice.Robot, :left_wheel, 5.0)  
iex> BB.Actuator.stop(Eunice.Robot, :left_wheel)
```

No wifi yet — leave the USB cable in. It's a network as well as a programmer.

Where you should be

- ☐ Did both tasks run without complaining?
- ☐ Is there firmware on the board, and does it boot?
- ☐ Can you reach it with `ssh nerves.local`?
- ☐ Do both wheels turn, and does each go the way you expect?



HOUR 4

Putting it all together

One more task and it stands up. Then we make it worth driving.

HOUR 4 • RUN THESE

Give it wifi, and a face

```
$ mix bb_nsk.add_display  
$ mix bb_nsk.add_wifi  
$ mix bb_nsk.add_web  
$ mix firmware && mix upload nerves.local
```

A face, its own access point, a dashboard, and a thumb pad for your phone.

HOUR 4 • LOOK AT IT

Tilt it and watch

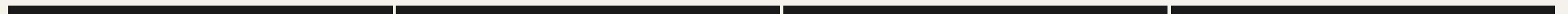
The panel on the front has the network name and the passphrase on it.

Join it, open the dashboard in a browser, then pick the robot up and lean it over.

What the loop gets to see

- The accelerometer gives you **down**. Hour 3 turned that into **lean**
- The gyro gives you **lean rate** — how fast it's tipping
- Both land together, a few hundred times a second
- Lean is also the **lever** — this robot drives by leaning

Two numbers in, one wheel command out. That's the whole controller.



The textbook wants four

YOU HAVE

- **lean** — from the IMU, through the filter
- **lean rate** — straight off the gyro

YOU DON'T

- where the wheels are
- how fast they're going

The two you have are the two that keep it upright. The other two would let it hold station as well.



P, I and D

- What you **want** minus what you've **got** is the **error**
- **P** — push in proportion to the error. Alone, it overshoots and rings
- **D** — ease off as the error shrinks. This is what damps the ringing
- **I** — sum the error over time, to cancel a standing offset

Most controllers use some subset of the three, and which subset is a design decision.

PD, with the I in an unusual place

- **P and D on the lean error** — the two gains you'll be tuning
- No I on the lean — the stubborn error is the **setpoint** itself
- So the **trim** sums the wheel command and slides the setpoint
- Deliberately slow — seconds, on a robot that falls in 70 ms

A trim that moves at the speed of the balance loop is just a badly tuned integrator — and it oscillates.

hour 4 • run this

Close the loop

```
$ mix bb_nsk.add_balance
```

The controller, the `:balancing` and `:fallen` states, the commands that drive them, and fifteen gains.

Fifteen numbers you can tune

```
controller :balancer,  
  {BB.NSK.Balance.Controller,  
    setpoint: param([:balance, :setpoint]),  
    proportional_gain: param([:balance, :proportional_gain]),  
    derivative_gain: param([:balance, :derivative_gain]),  
    catch_angle: param([:balance, :catch_angle]),  
    fall_angle: param([:balance, :fall_angle]),  
    # ...and ten more  
    drive_limit: param([:drive, :authority])}
```

Every one of them a parameter, because every one of them was found on the floor.

HOUR 4 • THE FAST LOOP

Build and upload

```
$ mix firmware  
$ mix upload nerves.local
```

Join the robot's wifi from your laptop first, or there's no `nerves.local` to upload to.

HOUR 4 • THE MOMENT

Stand it up

Open the dashboard on your phone. Hold the robot upright, hold it **still**, hit **arm**, and let go.

Arm is on the dashboard and on the drive page. No keyboard, no iex.

It doesn't know how fast it's going

- Standing still it's pretty good — friction does a lot of the work
- The gentle rocking is the **deadband**, not your gains
- But nothing measures speed, so nothing says "that's enough"
- Once it's properly moving it winds up rather than settling

Happy on a table. Just drive it somewhere you can catch it.

What to tune, and which way

IT'S DOING THIS	TRY
leaning away, slow to come back	P up
rocking, overshooting each way	D up
buzzing, wheels chattering	D down
sharp, twitchy, sawing	P down

One at a time, and treat it like a binary search. Halve it or double it, then narrow.

HOUR 4 • HAVE A PLAY

Tune something

Every parameter is on the dashboard. It's already standing — change one and watch.

Home is **P 180, D 4.0**. Try P at **240**, then **100**, then back.

HOUR 4 • THE OTHER THING YOU CAME FOR

Drive it

Same phone, same dashboard. Open the drive page and put your thumb on the pad.

Your repo, your robot



Everything you built today is yours, and there's plenty left to do.

github.com/beam-bots/bb_nsk

